

Spring Boot Auto-Configuration & Starter

1 What is Auto-Configuration?

Spring Boot auto-configuration automatically configures application components based on:

- Dependencies present in the classpath
- Application properties (e.g., `application.properties` / `yml`)
- Existing beans defined by the developer
- Environment & system configurations

INTERVIEW ANSWER

"Spring Boot auto-configuration automatically configures the application based on the dependencies and configuration available, reducing the amount of manual boilerplate configuration required."

2 How Does Auto-Configuration Work?

The core entry point is `@SpringBootApplication`, which combines three primary annotations:

```
@SpringBootApplication
|-- @Configuration (Declares Java-based config)
|-- @EnableAutoConfiguration (Triggers auto-config discovery)
    &boxbd;-- @ComponentScan (Scans for user beans)
```

`@EnableAutoConfiguration` instructs Spring Boot to inspect the classpath/environment and register auto-configuration classes loaded via Spring Boot's internal metadata mechanism.

3 Conditional Configuration

Auto-configuration evaluates conditional annotations before registering beans to avoid overriding user logic:

- `@ConditionalOnClass`: Applies configuration if specified classes are present on the classpath.
- `@ConditionalOnMissingBean`: Applies configuration only if no custom bean of that type exists.
- `@ConditionalOnProperty`: Applies configuration if a specific environment property is set.
- `@ConditionalOnBean`: Applies configuration if a required bean already exists.

```
JPA dependency on classpath? → Yes → Database connection properties set? → Yes → Custom DataSource bean
exists? → No → Auto-configure DataSource
```

4 Starter Dependencies

Starter are curated dependency bundles. Instead of managing multiple separate libraries, adding one starter brings in all required transitive dependencies.

Starter Dependency	Included Features & Purpose
<code>spring-boot-starter-web</code>	Spring MVC, RESTful Web Services, Jackson JSON, Embedded Tomcat
<code>spring-boot-starter-data-jpa</code>	Spring Data JPA, Hibernate ORM, AOP, JDBC API
<code>spring-boot-starter-data-mongodb</code>	MongoDB Java Driver, Spring Data MongoDB abstractions
<code>spring-boot-starter-security</code>	Spring Security core, web security configurations, authentication
<code>spring-boot-starter-validation</code>	Jakarta Bean Validation API, Hibernate Validator
<code>spring-boot-starter-test</code>	JUnit Jupiter, Mockito, AssertJ, Spring Test framework
<code>spring-boot-starter-actuator</code>	Application health checks, metrics, monitoring endpoints

5 Starter vs. Auto-Configuration

Dimension	Starter Dependency	Auto-Configuration
Core Role	Provides raw dependencies/jar libraries	Configures spring components based on dependencies
Answers	"What dependencies do I need?"	"How should Spring Boot configure those dependencies?"

Note: Starters do *not* automatically configure everything. For instance, `spring-boot-starter-data-mongodb` provides the MongoDB libraries, but you still need to configure server properties if not using local defaults.

6 How to Override or Disable Auto-Configuration

1. Declaring a Custom Bean: Because of `@ConditionalOnMissingBean`, declaring your own bean causes Boot's default to step back.

```
@Bean
public DataSource dataSource() {
    return new CustomDataSource(); // Disables default DataSource auto-configuration
}
```

2. Explicit Exclusion: Exclude specific auto-configuration classes directly via `@SpringBootApplication`:

```
@SpringBootApplication(exclude = {DataSourceAutoConfiguration.class})
public class Application { }
```

7 Key Cross-Questions & Solutions

Q1. What is the difference between Spring and Spring Boot?

Spring provides core framework features (IoC, DI, AOP). Spring Boot builds on top of Spring by providing opinionated defaults, starters, auto-configuration, and embedded servers to accelerate development.

Q2. What is the role of `@EnableAutoConfiguration`?

It instructs Spring Boot to automatically configure beans in the `ApplicationContext` by scanning the classpath and evaluating conditional annotations.

Q3. What is `@SpringBootApplication`?

It is a meta-annotation that combines `@Configuration`, `@EnableAutoConfiguration`, and `@ComponentScan`.

Q4. How does Spring Boot know what to auto-configure?

It evaluates the application classpath, properties files, existing beans, and registered conditional rules (`@ConditionalOnClass`, `@ConditionalOnMissingBean`).

Q5. Can we disable specific auto-configurations?

Yes, using the `exclude` property in `@SpringBootApplication` or setting `spring.autoconfigure.exclude` in configuration properties.

Master Cheat Sheet & 30-Second Interview Pitch

- **Starter:** Curated dependency bundle.
- **Auto-Configuration:** Smart configuration based on classpath & conditions.
- **@SpringBootApplication:** `@Configuration` + `@EnableAutoConfiguration` + `@ComponentScan`.
- **Starter ≠ Auto-Configuration:** Starter brings the jars; Auto-config sets them up.

30-Second Interview Summary:

"Spring Boot starters provide convenient dependency bundles for common features such as Web, JPA, MongoDB, Security, and Validation. Auto-configuration uses those dependencies, application properties, existing beans, and conditional configuration to automatically configure the application. `@SpringBootApplication` enables auto-configuration through `@EnableAutoConfiguration`. If we provide our own configuration, Boot's conditional auto-configuration backs off."